

Taller: Evaluación de Ancho de Banda y Latencia de Memoria

Benchmarks STREAM y multichase sobre Apple MacBook Air (chip M4)

Gabriela Hernández roncancio

17 de agosto de 2026

Índice

1. Conceptos Importantes	3
2. Especificaciones de la Máquina y Desarrollador	3
2.1. Características relevantes	3
2.2. Resumen de la jerarquía esperada	4
2.3. Software y configuración	5
3. STREAM	5
3.1. Proceso de instalación	5
3.2. Esquema de pruebas seguido para STREAM y justificación	6
3.3. Prueba A: perfil por nivel de caché	6
3.4. Resultados	7
3.5. Prueba B: escalabilidad con hilos	9
3.6. Ancho de banda teórico máximo y eficiencia alcanzada	10
4. Multichase	11
4.1. Contenedor	11
4.2. Entorno de ejecución medido	12
4.3. Parámetros y esquema de pruebas de multichase	13
4.4. Prueba 1: curva de latencia (variar el tamaño de los datos)	13
4.5. Transiciones de caché: lo medido frente a lo esperado	15
4.6. Prueba 2: efecto del patrón de acceso	16
4.7. Prueba 3: contención entre núcleos	17
5. Discusión: los dos benchmarks juntos	18
6. Limitaciones	19
7. Conclusiones	19

1. Conceptos Importantes

Para el desarrollo de este informe se consideró importante la aclaración de los siguientes conceptos.

- **Ancho de banda (bandwidth):** cantidad de datos que se pueden mover entre la memoria y el procesador por unidad de tiempo, típicamente medido en MB/s o GB/s.
- **Latencia de memoria:** tiempo que transcurre desde que el procesador solicita un dato hasta que lo recibe, medido en nanosegundos (ns) o ciclos de reloj.
- **Jerarquía de memoria (caché):** para reducir el impacto de la latencia y aumentar el ancho de banda efectivo, los procesadores modernos usan varios niveles de caché (L1, L2, L3/SLC) entre el núcleo y la memoria principal (RAM). Cada nivel es más rápido pero más pequeño que el siguiente.

Ambos benchmarks son necesarios porque miden propiedades opuestas del mismo subsistema:

Aspecto	STREAM	multichase
Métrica	Ancho de banda (MB/s): más es mejor	Latencia (ns): menos es mejor
Patrón de acceso	Secuencial y predecible	Dependiente (<i>pointer chasing</i>)
Prefetcher	Lo favorece: esconde la latencia	Lo neutraliza: expone la latencia
Tamaño de datos	Fijo y grande (evita la caché a propósito)	Variable: barre toda la jerarquía
Qué revela	El techo del bus de memoria	El tamaño y el costo de cada nivel de caché

2. Especificaciones de la Máquina y Desarrollador

Cuadro 1: Especificaciones de hardware de la plataforma evaluada

Componente	Descripción
Modelo	Apple MacBook Air (2024)
Procesador	Apple M4
Núcleos físicos	10 (4 de rendimiento + 6 de eficiencia)
Núcleos lógicos	10 (sin hyperthreading)
Memoria RAM	16 GB unificada
Arquitectura	ARM64 (Apple Silicon)

2.1. Características relevantes

Apple diseñó el chip M4 de forma tal que sucede lo siguiente.

- Los núcleos de rendimiento (P-cores) comparten una caché L2 masiva de hasta 16 MB.

- Los núcleos de eficiencia (E-cores), pensados para cargas ligeras y tareas de fondo, comparten una caché L2 propia y más pequeña, del orden de 4 MB, ya que sus cargas de trabajo requieren menos capacidad de caché.
- En lugar de una L3, Apple coloca una bolsa gigante de memoria ultra rápida al nivel de todo el chip llamada SLC (System Level Cache) o memory cache. Esta caché no solo la usa la CPU, sino también la GPU y el Neural Engine.

El siguiente es el diagrama oficial de Apple (de su Apple Silicon CPU Optimization Guide ¹, Figura 5.1) que muestra la arquitectura del "General Purpose Compute Complex".

Para conocer demás especificaciones de la máquina se ejecutaron los siguientes comandos.

```
1 sysctl -a | grep -E "hw.perflevel[0-1].l1dcachesize"
2 sysctl -a | grep -E "hw.perflevel[0-1].l2cachesize"
```

El resultado fue el siguiente.

```
1 hw.perflevel1.l1dcachesize: 65536 # L1d E-cores: 64 KB
2 hw.perflevel0.l1dcachesize: 131072 # L1d P-cores: 128 KB
3 hw.perflevel1.l2cachesize: 4194304 # L2 E-cores: 4 MB
4 hw.perflevel0.l2cachesize: 16777216 # L2 P-cores: 16 MB
```

2.2. Resumen de la jerarquía esperada

La siguiente tabla consolida los valores teóricos contra los cuales se contrastarán más adelante los "codos" medidos empíricamente con `multichase` (Sección 4.5).

Nivel	Tamaño	Origen del dato
L1D (P-core)	128 KiB	<code>hw.perflevel0.l1dcachesize</code> = 131072
L1D (E-core)	64 KiB	<code>hw.perflevel1.l1dcachesize</code> = 65536
L2 (clúster P)	hasta 16 MiB	Compartida por los 4 P-cores, no privada
L2 (clúster E)	≈4 MiB	Compartida por los 6 E-cores
SLC	no publicada	Apple no la expone; se estima empíricamente
RAM	16 GB LPDDR5X	Unificada, compartida con GPU y Neural Engine
Línea de caché	128 B	<code>hw.cachelinesize</code>
Página	16 KiB	Tamaño de página de Apple Silicon

¹Apple Developer, <https://developer.apple.com/download/apple-silicon-cpu-optimization-guide/>

2.3. Software y configuración

Elemento	Valor
Sistema operativo (anfitrión)	macOS 26.5.2 (build 25F84)
Compilador (STREAM, nativo)	Apple clang + libomp de Homebrew
Compilador (multichase, en contenedor)	gcc 13.3.0 (Ubuntu 13.3.0-6ubuntu2~24.04.1)
Plataforma de multichase	Docker Desktop 4.48.0, engine 28.5.1 (VM Linux)
Governor de CPU	No aplica: macOS gestiona la frecuencia de forma autónoma
Configuración NUMA	No aplica: SoC de un solo dominio de memoria unificada
Prefetcher de hardware	Activo y no deshabilitable desde el espacio de usuario

Estas tres últimas filas son relevantes para el taller: en Apple Silicon *no* existe un equivalente al `cpufreq governor` de Linux, ni interfaz para desactivar el prefetcher, ni topología NUMA que configurar. Por lo tanto, la frecuencia real durante las mediciones es desconocida y el prefetcher está siempre encendido; ambas cosas se retoman como limitaciones en la Sección 6.

3. STREAM

STREAM (John D. McCalpin, Universidad de Virginia) es el benchmark estándar de la industria para medir **ancho de banda sostenido de memoria principal**. Opera sobre tres arreglos grandes de tipo `double` (a , b , c) y ejecuta cuatro kernels (corre, uno detrás de otro, 4 bucles distintos, cada uno diseñado para ejercitar la memoria de una forma ligeramente diferente).

Kernel	Operación	Bytes por elemento movidos
Copy	$c_i = a_i$	16 (1 lectura + 1 escritura)
Scale	$b_i = \alpha \cdot c_i$	16 (1 lectura + 1 escritura)
Add	$c_i = a_i + b_i$	24 (2 lecturas + 1 escritura)
Triad	$a_i = b_i + \alpha \cdot c_i$	24 (2 lecturas + 1 escritura)

El ancho de banda reportado para cada kernel se calcula como:

$$\text{BW (MB/s)} = \frac{\text{bytes movidos} \times N}{\text{tiempo (s)} \times 10^6}$$

Para evaluar la velocidad de la RAM el arreglo debe ser tan grande como para que no quepa en ninguna caché (ni L1, ni L2, ni SLC). Si el arreglo cupiera en caché, STREAM estaría midiendo la velocidad de la caché en vez de la de la RAM, y el resultado no serviría para el objetivo de este informe.

3.1. Proceso de instalación

```
1 brew install libomp
2 #Descargar el código fuente de STREAM
3 curl -O https://www.cs.virginia.edu/stream/FTP/Code/stream.c
```

3.2. Esquema de pruebas seguido para STREAM y justificación

El esquema de pruebas se dividió en dos ejes independientes, cada uno diseñado para aislar una variable distinta:

- **Prueba A (perfil por nivel de caché):** variar el *tamaño de los datos* manteniendo el número de hilos fijo en 1, para observar cómo cambia el rendimiento según el dato quepa en L1, L2, SLC o tenga que ir a RAM.
- **Prueba B (escalabilidad con hilos):** fijar el tamaño de los datos en el nivel de RAM y variar el *número de hilos*, para observar en qué punto se satura el bus de memoria.

Separar estas dos pruebas es necesario porque STREAM solo tiene dos parámetros que controlan su comportamiento frente a la jerarquía de memoria: el tamaño del arreglo (STREAM_ARRAY_SIZE, fijo en tiempo de compilación) y el número de hilos (OMP_NUM_THREADS, variable en tiempo de ejecución). Si se variaran ambos a la vez no sería posible distinguir si un cambio en el rendimiento se debe al tamaño de los datos o al paralelismo.

3.3. Prueba A: perfil por nivel de caché

Esta prueba se enfoca en monitorear los límites de velocidad de las cachés sin interferencia de varios hilos durante el proceso; posteriormente se realizarán pruebas para analizar la saturación del canal mediante el cambio de hilos. Con base en los tamaños de caché confirmados con `sysctl`, se definió el N (STREAM_ARRAY_SIZE) a usar para apuntar a cada nivel de la siguiente forma.

Nivel objetivo	Tamaño caché aprox	Bytes objetivo	N	Footprint real
L1 (por P-core)	128 KB	~64 KB	2 500	60 KB
L2 (compartida P-cores)	hasta 16 MB	~8 MB	350 000	8.0 MB
SLC (memory cache)	≈ 24 MB	~20 MB	900 000	21.6 MB

$$\text{Footprint (bytes)} = 3 \times 8 \times N = 24 \times N$$

Para el nivel de RAM la idea original era emplear $N=100000000000$ sin embargo, era demasiado grande y se realizó hizo con $N=40000000$, el cual aún consiste un tamaño lo suficiente grande para salir del esquema de cache mostrado en la tabla anterior. Con $N = 40\,000\,000$ el footprint total es

$$24 \times 40\,000\,000 = 960\,\text{MB} \approx 916\,\text{MiB},$$

es decir **unas 57 veces** la L2 de 16 MiB del clúster de P-cores y muy por encima de cualquier estimación razonable de la SLC (la Sección 4.5 la acota entre 32 y 64 MiB). Para el proceso de compilación se utilizó el siguiente comando generado con la ayuda de IA (Claude Sonnet 5 Medium).

```

1 for tag in "l1:2500" "l2:350000" "slc:900000"; do
2     tag=${tag%%:*}
3     n=${tag##*:}
4     clang -Xpreprocessor -fopenmp "
5         -I/opt/homebrew/opt/libomp/include -L/opt/homebrew/opt/libomp
6         /lib -lomp "
7         -O3 -DSTREAMARRAYSIZE=$n -DNTIMES=50 "
8         stream.c -o stream$tag
9 done

```

Explicación de cada flag:

- `-Xpreprocessor -fopenmp -lomp`: habilita OpenMP usando `libomp` de Homebrew (clang de Apple no trae OpenMP nativo).
- `-O3`: optimización agresiva. Es obligatorio en STREAM porque sin optimizar el compilador no vectoriza los loops y se mediría el rendimiento del código sin optimizar, no de la memoria.
- `-DNTIMES=50`: número de repeticiones del bucle de medición (STREAM toma el mejor tiempo de las repeticiones para reducir ruido). Para los arreglos pequeños (L1/L2) se sube esto a 50 porque la ejecución es tan rápida que el reloj del sistema necesita más repeticiones para medir con precisión.

Para el caso del ejecutable de "streamRam exec" se utilizó el siguiente comando cambiando únicamente el `DNTIMES=50` a `DNTIMES=20` dado que `N` es un número más grande y un `DNTIMES` muy grande puede ralentizar el proceso.

```

1 clang -O3 -DSTREAMARRAYSIZE=40000000 -DNTIMES=20 "
2 -I/opt/homebrew/opt/libomp/include -L/opt/homebrew/opt/libomp/lib -
3 -lomp "
4 -Xpreprocessor -fopenmp stream.c -o streamram

```

Finalmente para almacenar los resultados de correr el conjunto de ejecutables recién generados en un solo archivo se empleó el siguiente comando.

```

1 for tag in l1 l2 slc ram; do
2     echo "=== $tag ===" >> resultadosstreamcache.log
3     OMPNUMTHREADS=1 ./stream$tag >> resultadosstreamcache.log
4 done

```

3.4. Resultados

Los resultados fueron los siguientes.

L1				
Function	Best Rate MB/s	Avg time	Min time	Max time
Copy:	inf	0.000001	0.000000	0.000005
Scale:	inf	0.000001	0.000000	0.000002
Add:	62914.6	0.000001	0.000001	0.000002
Triad:	62914.6	0.000001	0.000001	0.000002

L2				
Function	Best Rate MB/s	Avg time	Min time	Max time
Copy:	109757.5	0.000066	0.000051	0.000087
Scale:	79891.5	0.000081	0.000070	0.000090
Add:	80073.1	0.000120	0.000105	0.000135
Triad:	79173.4	0.000121	0.000106	0.000136

SLC				
Function	Best Rate MB/s	Avg time	Min time	Max time
Copy:	156471.4	0.000113	0.000092	0.000191
Scale:	116150.0	0.000142	0.000124	0.000232
Add:	120635.1	0.000206	0.000179	0.000270
Triad:	117353.6	0.000210	0.000184	0.000266

RAM				
Function	Best Rate MB/s	Avg time	Min time	Max time
Copy:	88020.3	0.007342	0.007271	0.007501
Scale:	85491.7	0.007579	0.007486	0.007893
Add:	93714.4	0.010430	0.010244	0.011592
Triad:	93740.6	0.010311	0.010241	0.010486

- El tamaño del arreglo (L1) es tan pequeño (2 500 elementos, solo 0.1 MiB en total) que el tiempo que tarda la CPU en procesarlo es inferior a 1 microsegundo.
- Hubo un comportamiento extraño dado que en la mayoría de operaciones la prueba con slc tuvo un mayor ancho de banda.
- El último caso de prueba con streamRam cumplió las expectativas de ser el más lento de todos.
- Hubo un comportamiento anormal en cuanto al orden de ancho de banda de las operaciones dado que lo esperado es que Copy y Scale tengan mayor ancho de banda que Add y Triad dado que manipulan menos arreglos, sin embargo, en el caso de las pruebas para L2 y SLC copy y ad tuvieron un mayor ancho de banda. Por último en el caso de RAM,

add y triad fueron las de mayor ancho de banda incluso aunque teóricamente debería ocurrir lo contrario.

3.5. Prueba B: escalabilidad con hilos

Usando únicamente el binario `stream_ram` (el que mide memoria principal real) compilado en la prueba A, se varió `OMP_NUM_THREADS` entre 1 y 10, el total de hilos disponibles en el M4 (4 P-cores + 6 E-cores, sin hyperthreading). La ejecución de las pruebas se llevó a cabo mediante el siguiente comando.

```
1 for T in 1 2 3 4 6 8 10; do
2     echo "=== hilos=$T ===" && streamscaling.log
3     OMPNUMTHREADS=$T ./streamram && streamscaling.log
4 done
```

Por qué solo con el binario de RAM y no con los de L1/L2/SLC: cada núcleo tiene su propia caché L1, así que escalar hilos sobre un binario pensado para L1 no tendría un significado claro (cada hilo seguiría viendo "su" L1). Lo mismo ocurre con el binario de L2 que se lo comparten por grupos (los P-cores y los E-cores). El fenómeno que interesa observar es la saturación de un recurso compartido lo cual solo aplica al bus de memoria hacia la RAM. **Por**

qué es necesario variar el número de hilos: un solo núcleo de procesador generalmente no puede generar suficientes peticiones de datos como para usar el 100 % del ancho de banda disponible de la RAM. Al agregar más hilos (vía OpenMP), se activan más núcleos simultáneos, cada uno generando sus propias peticiones de lectura/escritura, lo que va llenando el canal de memoria. No se puede saber de antemano en qué punto ese canal se satura: por eso se midió el rendimiento incrementando progresivamente los hilos (1, 2, 3, 4, 6, 8, 10), buscando el punto en el que agregar más hilos deja de traducirse en más ancho de banda (Lo que muestra una saturación del bus de memoria). Como se mostró en el anterior taller, en momentos de estrés el

sistema emplea los núcleos de rendimiento primero, y estos en particular tienen mayor caché y frecuencia máxima que los núcleos de eficiencia, lo cual anteriormente llevó a concluir en la vital importancia de observar el comportamiento conforme se agregan los núcleos de eficiencia a la ecuación, en este caso como hilos. **Resultados:**

Hilos	Copy	Scale	Add	Triad
1	87575.2	85288.0	93731.8	93777.7
2	101911.7	99734.5	96057.3	96229.5
3	103228.5	101010.5	97165.3	97193.5
4	103161.1	101407.4	97245.1	97186.5
6	107006.1	97013.2	97560.9	97591.6
8	107044.5	97726.6	97431.0	97402.7
10	107074.4	96010.4	97423.9	97452.2

Con base a los resultados, da igual si se ponen a trabajar 4, 6, 8 o 10 núcleos en paralelo; la tasa de transferencia se mantiene cerca de los ~ 107 GB/s. Esto demuestra empíricamente la

contención entre núcleos: los núcleos adicionales pasan la mayor parte del tiempo esperando en fila a que el bus de memoria unificada les despache los datos, confirmando que agregar más poder de cómputo no sirve de nada si la tubería física de la memoria ya está llena. Con 1 y 2 hilos, Copy y Scale son las funciones más rápidas. Para hilos 6 y 10, Add y Triad terminan siendo más rápidos que Scale, mostrando que estas dos operaciones se benefician de durar más tiempo realizando cálculos matemáticos que esperando para acceder a RAM.

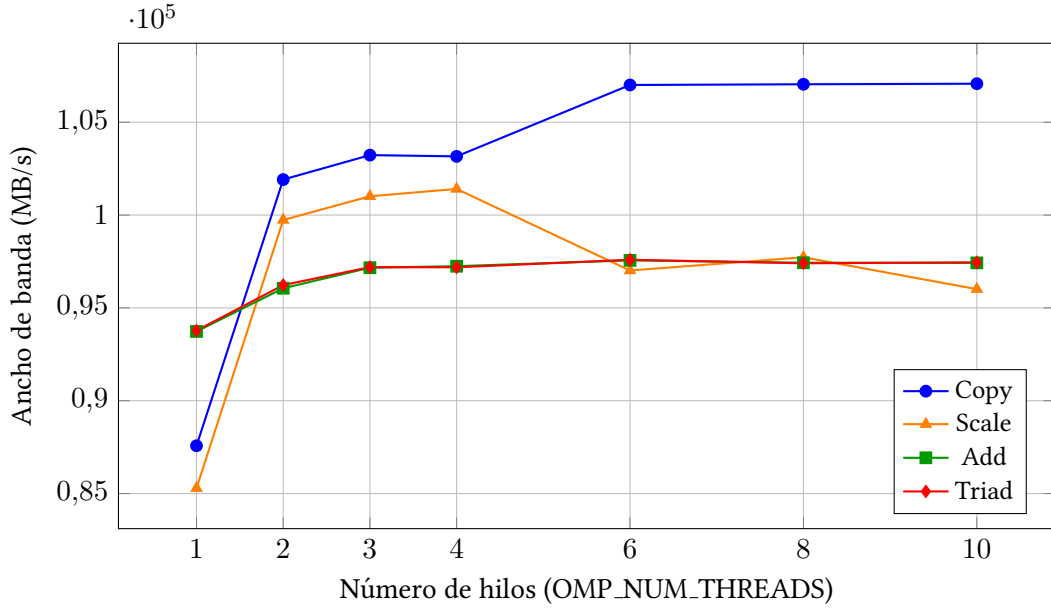


Figura 1: Escalabilidad del ancho de banda de STREAM (binario RAM) al variar el número de hilos (OMP_NUM_THREADS). Se observa un crecimiento marcado de 1 a 2–3 hilos, seguido de una meseta desde 3–4 hilos en adelante, evidencia de saturación del bus de memoria unificada.

3.6. Ancho de banda teórico máximo y eficiencia alcanzada

El ancho de banda pico de una memoria se calcula a partir del ancho del bus y de la tasa de transferencia:

$$BW_{\text{teórico}} = \frac{\text{ancho del bus (bits)}}{8} \times \text{tasa de transferencia (MT/s)}$$

El M4 base integra memoria **LPDDR5X-7500** sobre una interfaz de **128 bits**, organizada como 8 canales de 16 bits:

$$BW_{\text{teórico}} = \frac{128}{8} \times 7500 \frac{\text{MT}}{\text{s}} = 16 \text{ B} \times 7500 \frac{\text{MT}}{\text{s}} = 120\,000 \frac{\text{MB}}{\text{s}} = 120 \text{ GB/s}$$

Contrastando con el mejor valor sostenido medido (Copy con 6 o más hilos, 107 074,4 MB/s):

$$\eta = \frac{107\,074,4}{120\,000} \times 100 = \mathbf{89,2\%}$$

Kernel	Mejor medido (MB/s)	Eficiencia vs. 120 GB/s
Copy	107 074,4	89,2 %
Scale	101 407,4	84,5 %
Add	97 560,9	81,3 %
Triad	97 591,6	81,3 %

Lectura del resultado. Una eficiencia del 89 % es muy alta: la literatura sitúa a STREAM típicamente entre el 60 y el 85 % del pico teórico en plataformas con DRAM convencional, porque el refresco de la DRAM, los cambios de fila (*row activation*) y la alternancia entre lecturas y escrituras impiden sostener el máximo.

4. Multichase

Multichase (desarrollado por Google) mide **latencia de acceso a memoria** mediante *pointer chasing*: construye una lista enlazada dentro de un buffer de tamaño N , con un patrón de saltos (stride) configurable, y recorre la lista siguiendo el puntero `next` de cada nodo. Como cada acceso depende del resultado del anterior, el procesador **no puede prefetchear ni paralelizar la cadena de dependencias** (a diferencia de STREAM, que es fácilmente prefetcheable por ser acceso secuencial). Esto hace que multichase mida la *latencia real* de la jerarquía de memoria, sin que el prefetcher de hardware “esconda” la latencia.

4.1. Contenedor

Debido a que multichase emplea llamadas al sistema propias de Linux (en particular `sched_setaffinity`, las macros `CPU_SET` y soporte para *hugepages*), y estas no están disponibles de forma nativa en macOS, se optó por ejecutar el benchmark dentro de un contenedor Docker. Esto significa que multichase no interactúa directamente con el sistema operativo (macOS), sino con el kernel de la máquina virtual Linux ligera que Docker Desktop levanta internamente para dar soporte a los contenedores. El contenedor se inició con el siguiente comando.

```
1 docker run -it --name taller-multichase --privileged ubuntu:24.04
  bash
```

Listing 1: Creación del contenedor Docker para ejecutar multichase

Donde:

- `--privileged`: necesario porque multichase intenta utilizar *hugepages* y algunas llamadas de bajo nivel; sin este flag, algunas mediciones pueden fallar silenciosamente.
- `ubuntu:24.04`: en Docker Desktop para Mac con Apple Silicon, esta imagen se descarga automáticamente en su variante `arm64` (misma arquitectura que el chip M4 del equipo de prueba), por lo que no es necesario especificar la plataforma de forma explícita ni se incurre en emulación x86.

dentro del contenedor se corrió lo siguiente.

```
1 apt update && apt install -y build-essential git git clone https://  
  github.com/google/multichase.git cd multichase make
```

Para automatizar el barrido completo (39 + 22 + 8 + 20 mediciones) el contenedor se relanzó en segundo plano con `-dit`, de modo que cada punto pudiera lanzarse con `docker exec` sin necesidad de una sesión interactiva:

```
1 docker run -dit --name taller-multichase --privileged ubuntu:24.04  
  bash  
2 docker exec taller-multichase bash -lc "  
3   apt-get update -qq && "  
4   apt-get install -y -qq build-essential git && "  
5   cd /root && "  
6   ( [ -d multichase ] — git clone https://github.com/google/  
   multichase.git ) && "  
7   cd multichase && make"
```

4.2. Entorno de ejecución medido

Elemento	Valor observado
Anfitrión	MacBook Air, Apple M4, macOS 26.5.2 (build 25F84)
Plataforma de ejecución	Docker Desktop 4.48.0, engine 28.5.1 (VM Linux)
Imagen del contenedor	ubuntu:24.04, creado con <code>--privileged</code>
Arquitectura	aarch64 (nativa, sin emulación)
nproc	10
lscpu: CPU(s)	10 (lista en línea 0–9)
lscpu: Vendor ID	Apple
lscpu: Model name	– (no expuesto a la VM)
lscpu: Thread(s) per core	1
lscpu: Core(s) per cluster	10
lscpu: Cluster(s)	1
lscpu: BogomIPS	48,00
MemTotal (VM)	8 025 424 kB ($\approx$ 7,65 GiB)
Compilador	gcc 13.3.0 (Ubuntu 13.3.0-6ubuntu2~24.04.1)
Commit de multichase	8cc8681
Banderas de compilación	-O3 -march=armv8.1-a+lse -static -pthread
Fecha de ejecución	14 de agosto de 2026, 19:52–20:07 (UTC–05)

Antes de medir se cerraron las aplicaciones no esenciales (System Settings, Safari, Music, WhatsApp, Preview, Acrobat, GitHub Desktop, Console). Además, se ejecutaron en serie, no en paralelo.

4.3. Parámetros y esquema de pruebas de multichase

Flag	Significado	Uso en este esquema
-m	Tamaño del buffer (4k, 8m, 1g)	Variable principal de la Prueba 1
-s	Stride en bytes entre accesos	Variable principal de la Prueba 2
-t	Número de hilos	Variable principal de la Prueba 3
-n	Número de muestras de 0,5 s	Fijo en 10 (5 s por punto)

El esquema consta de **cuatro pruebas**, cada una de las cuales varía *una sola* variable y mantiene fijas las demás, de modo que cualquier cambio en la latencia sea atribuible sin ambigüedad a esa variable:

#	Variable que cambia	Variabes fijas	Pregunta que responde
1	Tamaño (-m)	stride, hilos	¿Dónde están las fronteras de L1, L2, SLC y RAM?
2	Stride (-s)	tamaño, hilos	¿Cuánto influyen el prefetcher y el TLB?
3	Hilos (-t)	tamaño, stride	¿Cuánto se degrada la latencia por contención?

4.4. Prueba 1: curva de latencia (variar el tamaño de los datos)

Idea de la prueba. Mientras los datos quepan completos en un nivel de caché, todos los accesos son aciertos en ese nivel y la latencia se mantiene baja y constante. Cuando los datos ya no caben, el procesador tiene que ir al siguiente nivel, que es más lento, y la latencia sube de golpe. Ese salto brusco se llama **codo**, y su posición indica hasta dónde llega la capacidad de ese nivel.

Se midieron 39 tamaños distintos, desde 4 KiB hasta 1 GiB, con **un solo hilo** y un **stride de 128 bytes**. El *stride* es la distancia en bytes entre un acceso y el siguiente; 128 B es justo el tamaño de una línea de caché en el M4, así que cada acceso cae en una línea distinta y ninguno se abarata reutilizando la línea anterior. Se usa un solo hilo para que el único factor que cambie sea el tamaño.

```
1 docker exec taller-multichase bash -lc 'cd /root/multichase && "  
2 echo "sizebytes,sizelabel,latencians" ; /root/p1tamano.csv && "  
3 for S in 4k 8k 16k 32k 64k 96k 128k 160k 192k 256k 384k 512k 768k "  
4     1m 2m 3m 4m 6m 8m 10m 12m 14m 16m 18m 20m 24m 28m 32m "  
5     40m 48m 64m 96m 128m 192m 256m 384m 512m 768m 1g; do "  
6     L=$(./multichase -m $S -s 128 -n 10); "  
7     B=$(numfmt --from=iec $-S ) ; "  
8     echo "$B,$S,$L" ; /root/p1tamano.csv; "  
9 done'
```

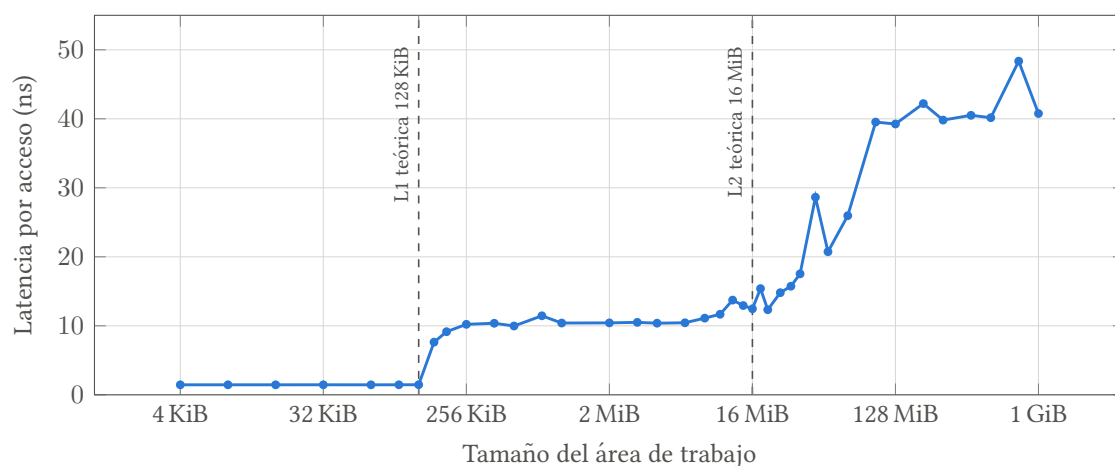


Figura 2: Latencia por acceso según el tamaño de los datos (1 hilo, stride 128 B). El eje X está en escala logarítmica porque el rango va de 4 KiB a 1 GiB. Las líneas punteadas marcan los tamaños de caché que reporta el sistema.

La curva tiene cuatro zonas bien diferenciadas:

Zona	Rango	Latencia media	Qué significa
1 (L1)	4 KiB – 128 KiB	1,46 ns	Todo cabe en la caché más cercana al núcleo
2 (L2)	160 KiB – 6 MiB	10,09 ns	Ya no cabe en L1; se resuelve en L2
3 (subida)	8 MiB – 32 MiB	13,77 ns	Se sale de L2 y entra en juego la SLC
4 (RAM)	96 MiB – 1 GiB	41,32 ns	Ninguna caché alcanza; todo va a memoria

Entre la zona 1 y la zona 4 la latencia empeora **28 veces** (de 1,46 a 41,32 ns). Ese es, en una sola cifra, el costo de salirse de la caché.

Tabla de datos completa (los 39 puntos medidos)

Tamaño	ns	Tamaño	ns	Tamaño	ns
4k	1,451	768k	11,454	28m	15,739
8k	1,454	1m	10,405	32m	17,532
16k	1,453	2m	10,427	40m	28,634
32k	1,455	3m	10,502	48m	20,739
64k	1,451	4m	10,383	64m	25,962
96k	1,456	6m	10,441	96m	39,527
128k	1,464	8m	11,114	128m	39,250
160k	7,633	10m	11,677	192m	42,207
192k	9,148	12m	13,728	256m	39,815
256k	10,217	14m	12,939	384m	40,508
384k	10,371	16m	12,470	512m	40,160
512k	9,974	18m	15,394	768m	48,358
		20m	12,332	1g	40,766
		24m	14,803		

4.5. Transiciones de caché: lo medido frente a lo esperado

Nivel	Esperado	Medido	Comentario
L1 (P-core)	128 KiB	128 KiB	Coincide exactamente. El último punto plano es 128 KiB (1,464 ns) y el siguiente, 160 KiB, ya vale 7,633 ns.
L2	hasta 16 MiB	6–8 MiB	Menor que lo esperado: la curva se mantiene plana solo hasta 6 MiB y empieza a subir en 8 MiB. En la práctica, un solo hilo aprovecha menos de la mitad de la L2.
SLC	no publicada	entre 32 y 64 MiB (aprox.)	No hay un salto único, sino una subida progresiva. Ver comentario abajo.
RAM	16 GB	desde 96 MiB	La latencia se estabiliza alrededor de 41 ns y ya no cambia hasta 1 GiB.

Por qué la SLC no se puede medir con precisión. Se esperaba un último salto limpio que marcara el final de la SLC. En lugar de eso, la curva sube *poco a poco* entre 8 MiB (11,1 ns) y 96 MiB (39,5 ns), e incluso retrocede en algunos puntos (20 MiB mide menos que 18 MiB). Si se toma el último salto grande, 64 → 96 MiB, la SLC estaría entre 32 y 64 MiB. Pero una frontera de caché real produce un escalón, como el de la L1, y aquí hay una rampa. La explicación más probable es que **la SLC no la usa solo la CPU**: también la comparten la GPU y el Neural Engine, de modo que no se llena de golpe, sino que va perdiendo eficacia gradualmente. Con este experimento no se puede dar un número exacto; haría falta medir muchos más tamaños entre 32 y 96 MiB y repetir cada punto varias veces.

4.6. Prueba 2: efecto del patrón de acceso

Idea de la prueba. Aquí se deja fijo el tamaño de los datos y se cambia el *stride*, es decir, qué tan separados están entre sí los accesos. Se hicieron dos barridos: uno con 8 MiB (datos que están dentro de la caché) y otro con 256 MiB (datos que están en memoria principal), para ver si el patrón de acceso pesa más en un caso que en el otro.

```
1 docker exec taller-multichase bash -lc 'cd /root/multichase && "
2 echo "tamano, stridebytes, latencians" && /root/p2stride.csv && "
3 for M in 8m 256m; do "
4   for ST in 16 32 64 128 256 512 1024 2048 4096 8192 16384; do "
5     L=$(./multichase -m $M -s $ST -n 10); "
6     echo "$M,$ST,$L" && /root/p2stride.csv; "
7   done; "
8 done'
```

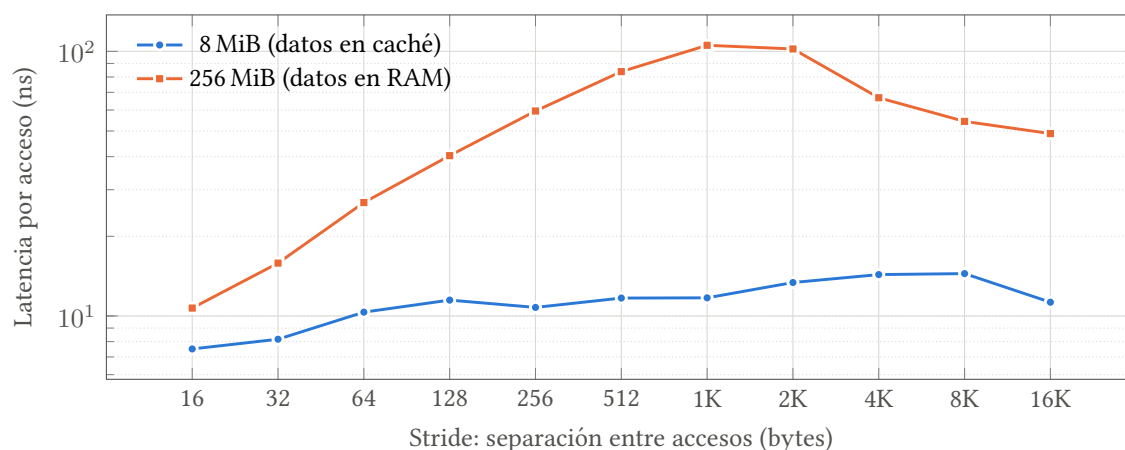


Figura 3: Latencia según la separación entre accesos. La serie de RAM varía casi 10 veces de un extremo al otro; la de caché apenas 2 veces.

Stride (B)	8 MiB (caché)		256 MiB (RAM)	
	ns	veces vs. 128 B	ns	veces vs. 128 B
16	7,505	0,65	10,710	0,27
32	8,167	0,71	15,838	0,39
64	10,336	0,90	26,829	0,66
128	11,480	1,00	40,350	1,00
256	10,772	0,94	59,499	1,47
512	11,684	1,02	83,820	2,08
1024	11,708	1,02	105,400	2,61
2048	13,378	1,17	102,100	2,53
4096	14,333	1,25	66,784	1,66
8192	14,457	1,26	54,350	1,35
16384	11,272	0,98	48,897	1,21

Stride pequeño (16–64 B): la latencia parece mejor de lo que es. Con stride de 16 B, ocho accesos consecutivos caen dentro de la misma línea de 128 B: solo el primero paga el costo de traerla, los otros siete la encuentran ya cargada.

Stride de 128 B: la medida limpia. Cada acceso cae en una línea nueva, así que se paga un fallo completo por acceso.

Stride grande: un resultado inesperado. Lo esperado era que a mayor separación la latencia siguiera subiendo. Lo que ocurre es que sube hasta 1 KiB (105,4 ns) y a partir de ahí **baja** hasta los 48,9 ns del stride de 16 KiB.

4.7. Prueba 3: contención entre núcleos

Idea de la prueba. Se repite la misma medición (256 MiB, stride 128) pero con varios hilos trabajando al mismo tiempo, de 1 a 10. Si todos compiten por el mismo bus de memoria, las peticiones se hacen fila y cada acceso individual tarda más. Es el complemento de lo que se midió con STREAM: allí se vio cuánto ancho de banda total se consigue; aquí se ve cuánto le cuesta a cada acceso.

```
1 docker exec taller-multichase bash -lc 'cd /root/multichase && "  
2 echo "hilos,latencias" & /root/p3hilos.csv && "  
3 for T in 1 2 3 4 5 6 8 10; do "  
4   L=$(./multichase -m 256m -s 128 -t $T -n 10); "  
5   echo "$T,$L" && /root/p3hilos.csv; "  
6 done'
```

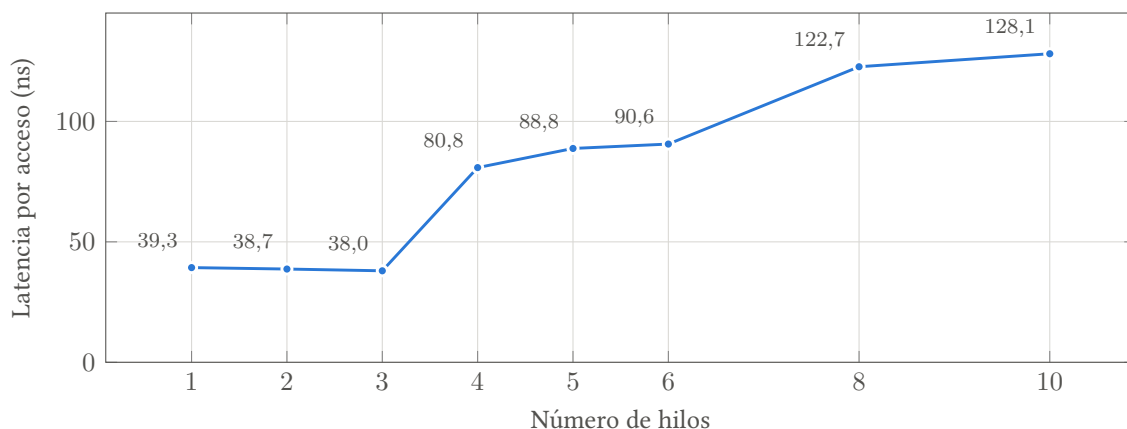


Figura 4: Latencia por acceso según el número de hilos que trabajan a la vez. Es plana hasta 3 hilos y se duplica al pasar a 4.

Hilos	Latencia (ns)	veces vs. 1 hilo	Cambio respecto al punto anterior
1	39,309	1,00	—
2	38,714	0,98	−1,5 %
3	37,984	0,97	−1,9 %
4	80,827	2,06	+112,8 %
5	88,778	2,26	+9,8 %
6	90,602	2,30	+2,1 %
8	122,700	3,12	+35,4 %
10	128,100	3,26	+4,4 %

Coincide con la saturación vista en STREAM. Hasta 3 hilos la latencia no se degrada nada (baja un 1,9 %, que está dentro del ruido); a partir de 4 se dispara. Es exactamente el mismo punto donde STREAM dejó de ganar ancho de banda. Los dos experimentos describen el mismo fenómeno: hasta 3 hilos la memoria atiende todas las peticiones sin hacer esperar a nadie; pasado ese punto se forma una fila, el ancho de banda total ya no crece y el costo lo paga cada acceso individual. Con 10 hilos, cada acceso cuesta 3,26 veces lo que costaba con uno.

5. Discusión: los dos benchmarks juntos

Nivel	Latencia (multichase)	Ancho de banda (STREAM, Copy)	Comentario
L1	1,45 ns	no medible	STREAM es demasiado rápido aquí para el reloj
L2	10,4 ns	≈110 GB/s	7 veces la latencia de L1
SLC	12–26 ns	≈156 GB/s	Subida gradual, no escalón
RAM	39,1 ns	107 GB/s (saturado)	27 veces la latencia de L1

Latencia y ancho de banda no empeoran al mismo ritmo. De L1 a RAM la latencia se multiplica por 27, mientras que el ancho de banda apenas cae un 30 %. La razón es el prefetcher: cuando los accesos son secuenciales y predecibles, el procesador pide muchos datos por adelantado y a la vez, de modo que la espera de cada acceso individual queda oculta. Cuando cada acceso depende del anterior eso es imposible. Por eso un programa con buen patrón de acceso puede trabajar casi al máximo de la memoria, y otro que persigue punteros (listas enlazadas, árboles, tablas hash dispersas) puede ser decenas de veces más lento *en la misma máquina*.

Los dos benchmarks encuentran el mismo punto de saturación. Este es el resultado más sólido del taller:

	STREAM (ancho de banda)	multichase (latencia)
1–3 hilos	Crece de 87,6 a 103,2 GB/s	Plana: de 39,3 a 38,0 ns
4 hilos	Deja de crecer (103,2 GB/s)	Se duplica: 80,8 ns
6–10 hilos	Se mantiene en ≈107 GB/s	Sigue subiendo hasta 128,1 ns

Ambas curvas cambian de comportamiento en el mismo punto, entre 3 y 4 hilos, a pesar de haberse medido con programas distintos, compiladores distintos y sistemas distintos (macOS

nativo frente a una máquina virtual Linux). Es la misma barrera física vista desde dos lados: mientras la memoria no está saturada, agregar hilos aporta más ancho de banda sin costo en latencia; una vez saturada, lo único que puede crecer es la fila de espera.

6. Limitaciones

1. **multichase no midió el chip directamente, sino una máquina virtual sobre el chip.** Todo corrió como proceso Linux dentro de la VM de Docker Desktop. Cada acceso a memoria pasa por una capa extra de traducción de direcciones, lo que afecta sobre todo a la Prueba.
2. **No se pudo elegir en qué tipo de núcleo corre cada hilo.** Dentro del contenedor, el sistema reporta 10 núcleos iguales; la división real entre núcleos de rendimiento y de eficiencia no se ve. Por eso toda la interpretación P/E de la Sección 4.7 es una hipótesis razonable, no una observación.
3. **Condiciones de fondo controladas, no eliminadas.** Se cerraron las aplicaciones pesadas, pero macOS mantiene decenas de procesos activos. Además, la SLC y la memoria son compartidas con la GPU y el Neural Engine, y esa competencia no se puede controlar; contribuye a la forma de rampa de la Sección 4.5.

7. Conclusiones

1. **El ancho de banda sostenido del M4 es de ≈ 107 GB/s, el 89 % del máximo teórico de 120 GB/s.** Es una eficiencia alta, favorecida por tener la memoria integrada en el mismo paquete que el procesador y por los 8 canales del controlador.
2. **La memoria se satura con solo 3 o 4 hilos de los 10 disponibles.** A partir de ahí, agregar núcleos no aporta ancho de banda y sí empeora la latencia: con 10 hilos cada acceso cuesta 3,26 veces lo que costaba con uno. Para un programa limitado por memoria, paralelizar más allá de 4 hilos en esta máquina no ayuda.
3. **La jerarquía de memoria abarca un rango de 27 veces en latencia,** de 1,45 ns en L1 a 39,1 ns en RAM. La frontera de L1 medida coincide exactamente con la que reporta el sistema (128 KiB), lo que valida el método.
4. **Los tamaños de la hoja de especificaciones no son los tamaños útiles.** La L2 de 16 MiB se comporta como 6–8 MiB para un solo hilo, y la SLC no produce una frontera nítida sino una subida gradual, porque la comparte con la GPU y el Neural Engine. Dimensionar bloques de trabajo con los valores nominales lleva a errores, como ocurrió con la corrida “L2” de STREAM.
5. **El patrón de acceso pesa tanto como la jerarquía.** Los mismos 2 MiB de datos cuestan 10,4 ns por acceso si están contiguos y 48,9 ns si están dispersos en muchas páginas distintas. Y el contraste entre STREAM (107 GB/s en acceso secuencial) y multichase (39 ns

por acceso dependiente) muestra que el prefetcher es lo que separa un programa rápido de uno lento sobre el mismo hardware.